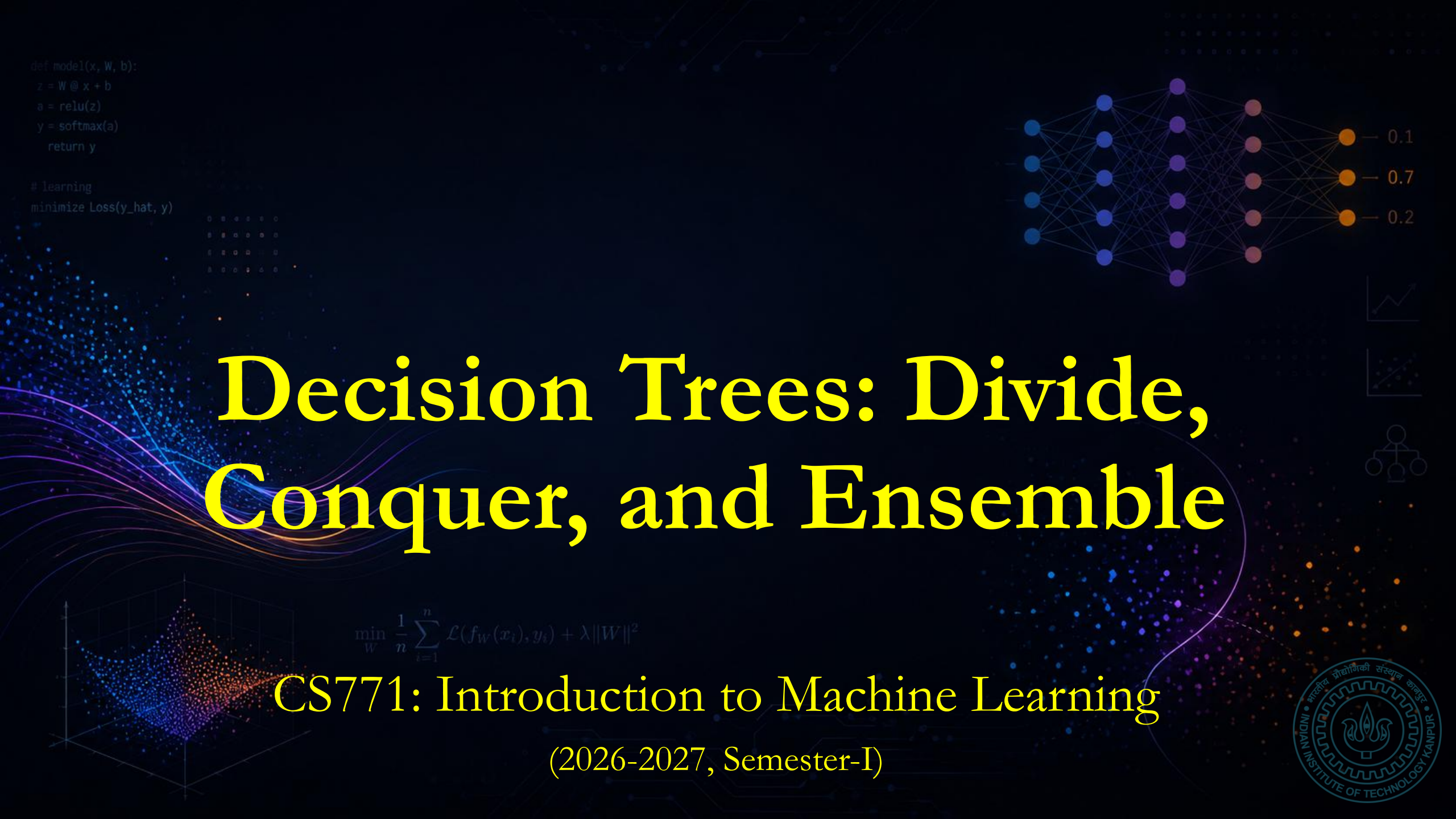
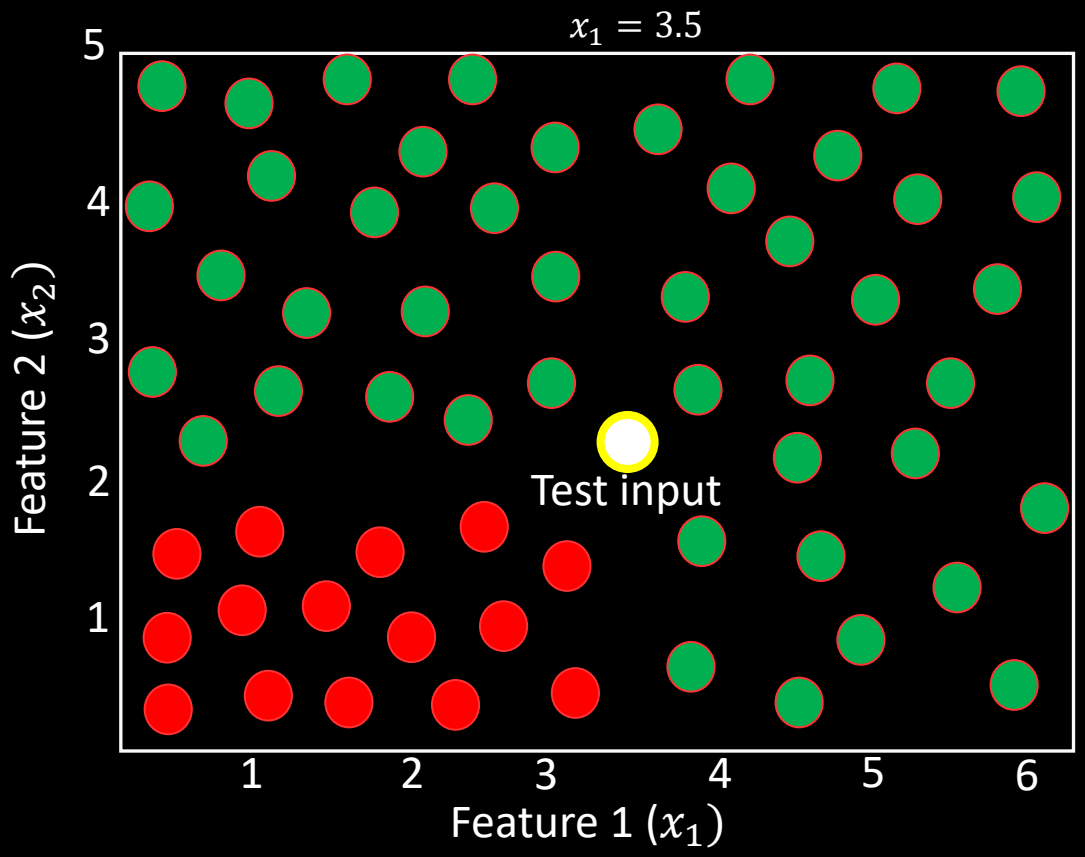


Decision Trees: Divide, Conquer, and Ensemble

CS771: Introduction to Machine Learning

(2026-2027, Semester-I)

K-NN: Prediction Speed Bottleneck



- The dataset has 60 training inputs
- Naïve K-NN: 60 distance computations for each test input (and sorting the distances to find the nearest neighbors)
- A faster alternative: Divide and conquer
  - Decision Trees follow this very idea and can give blazing fast predictions!

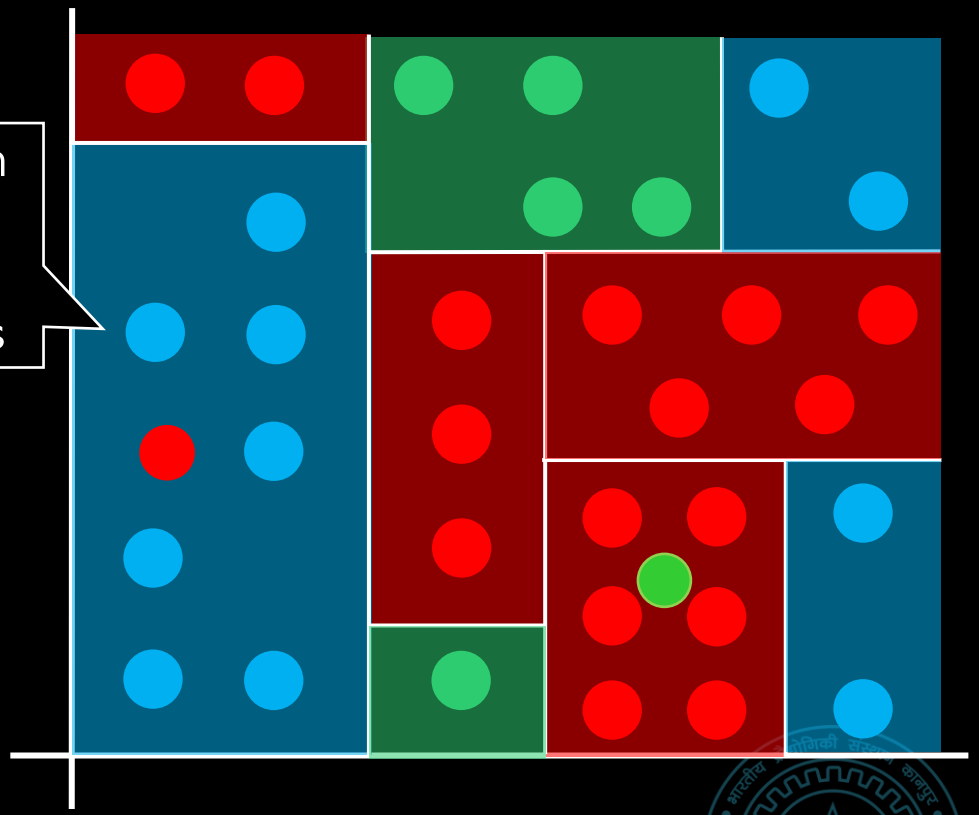
A Simple Divide & Conquer Scheme

Divide: Recursively splitting the input space till we get small homogeneous regions

Conquer: If a test input's region can be quickly located, prediction is fast

Some "design" decisions worth considering here:

- Where should we split at each stage?
- What rule to use for a split?
- When should we stop splitting any further?
- How homogeneous is "homogeneous enough"?
- What prediction rule to use within each region?



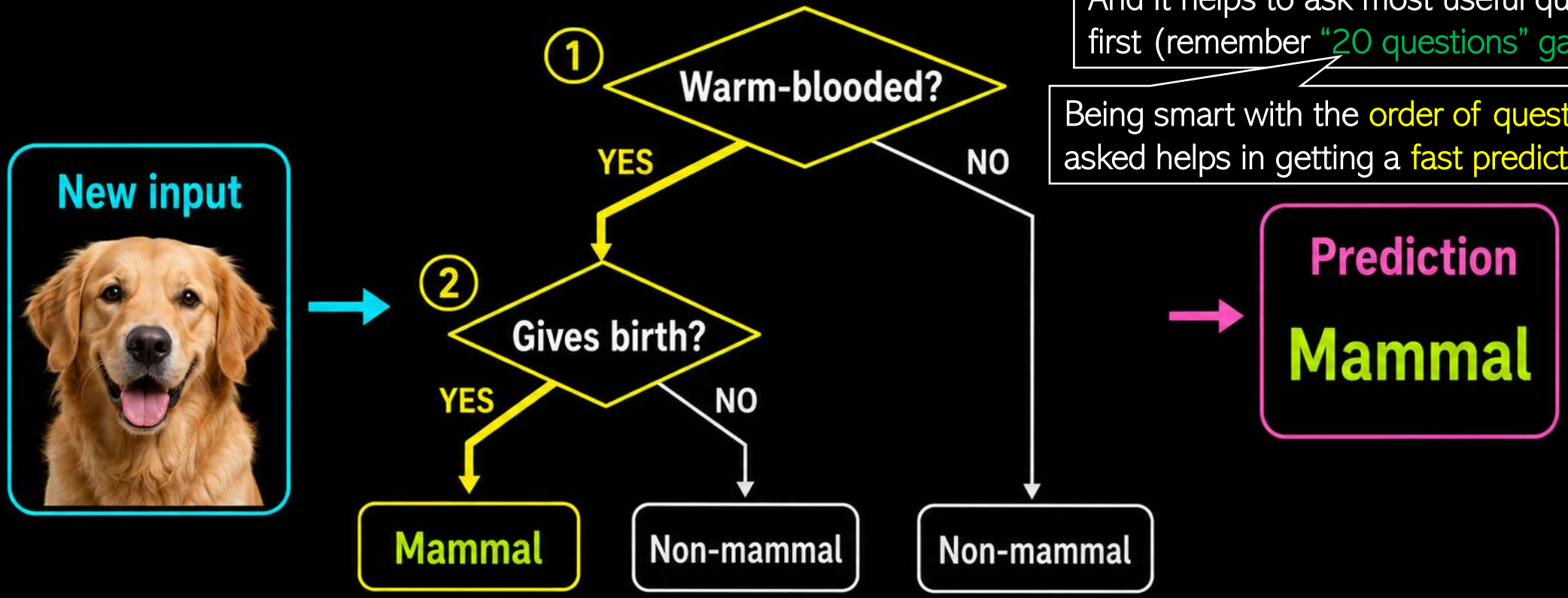
These "design" decisions are made using the training data

Decision Trees Naturally Solve Divide & Conquer

Decision Trees solve divide and conquer by asking a series of questions

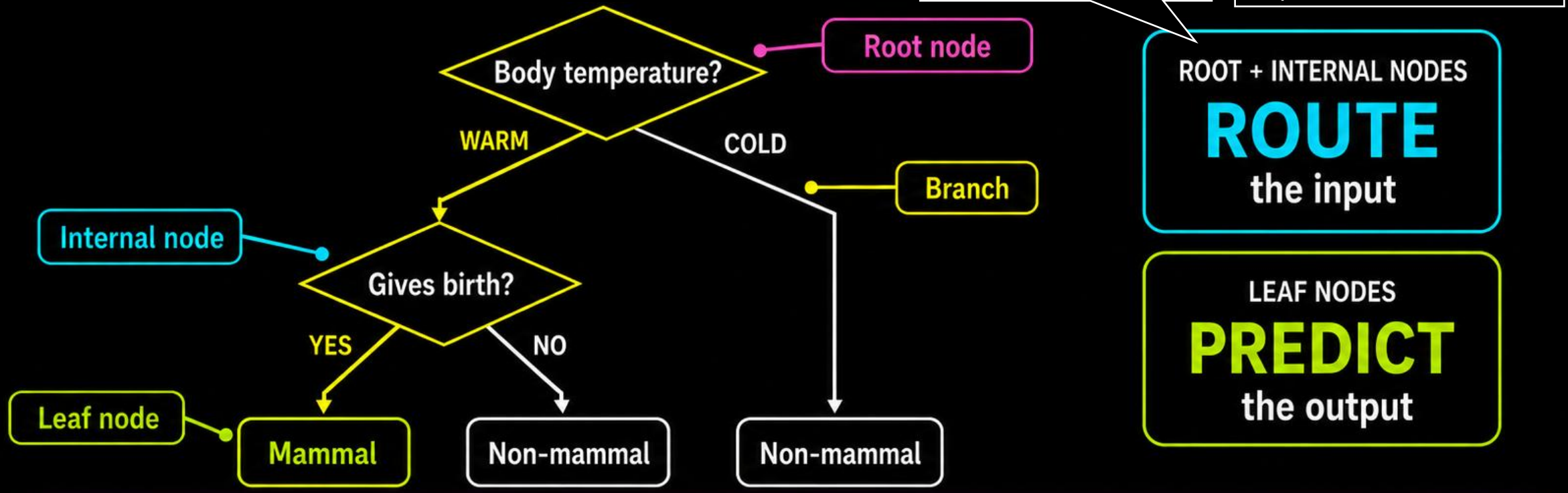
And it helps to ask most useful questions first (remember "20 questions" game?)

Being smart with the order of questions asked helps in getting a fast prediction



Each answer chooses the next question — until a prediction is reached.

Anatomy of a Decision Tree



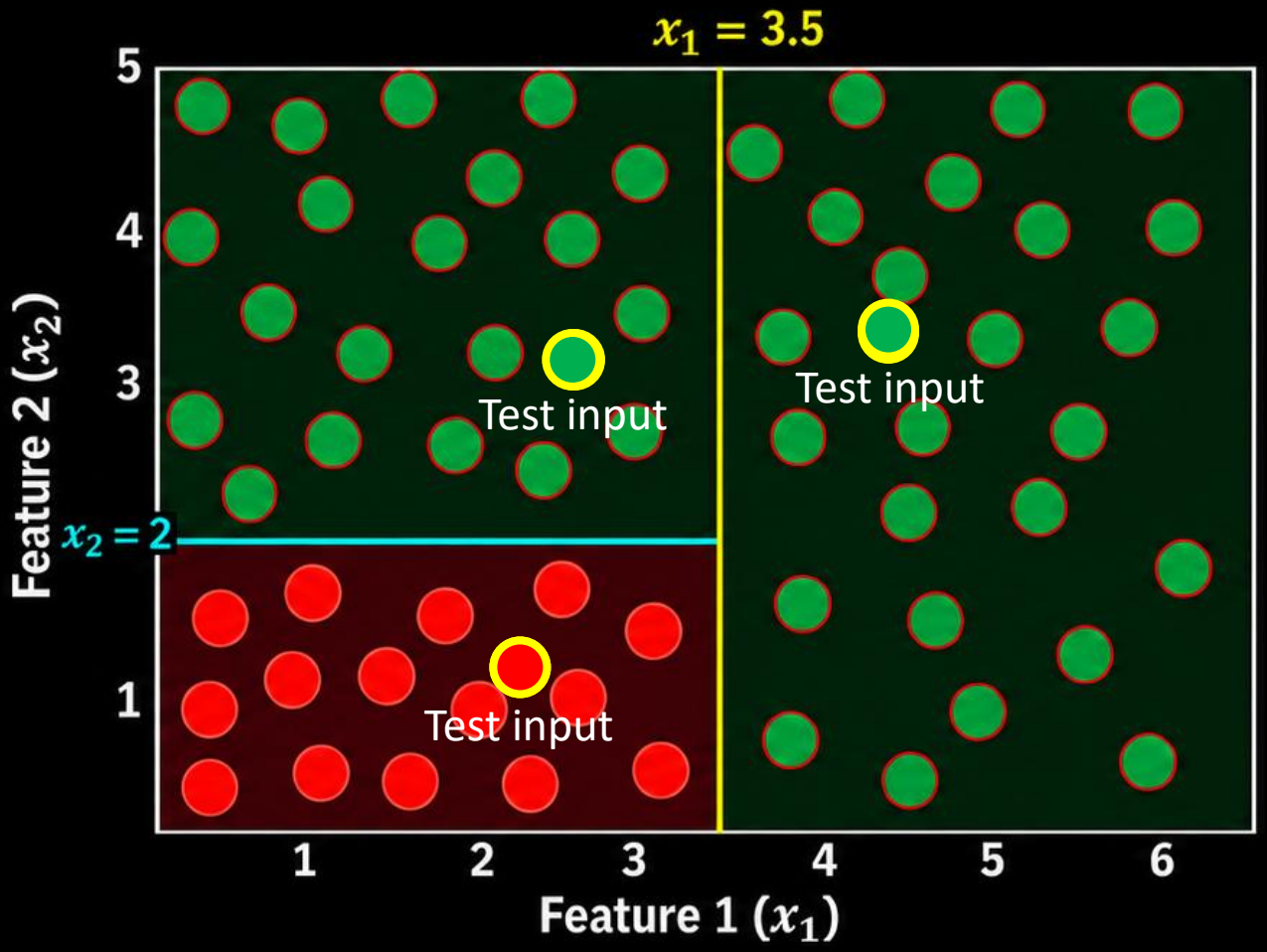
Routers and **predictors** can be simple rules or themselves be **learned ML models**

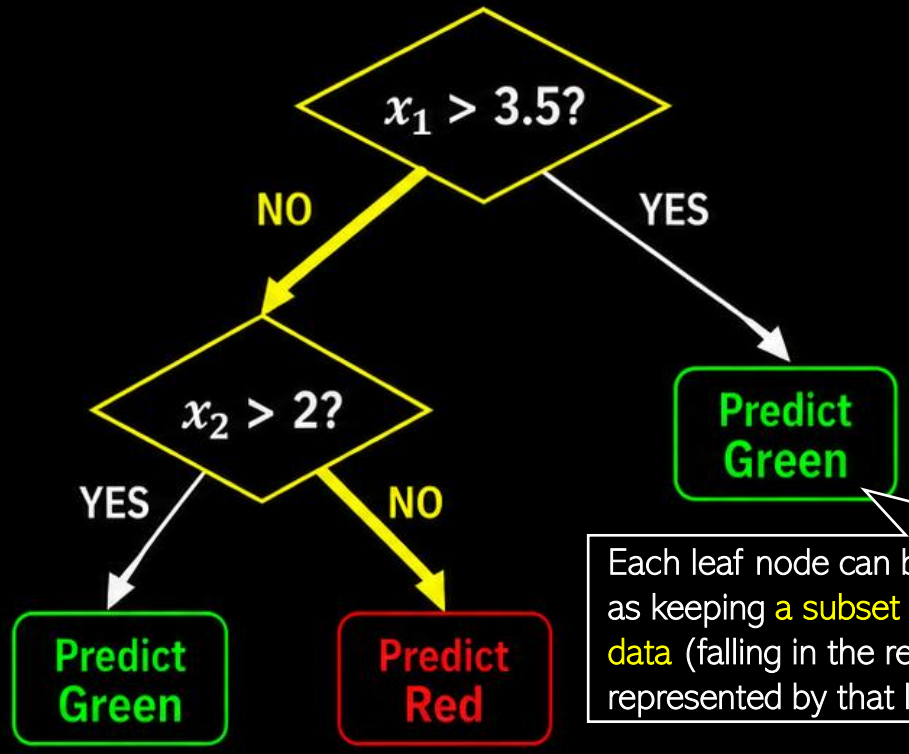
A decision tree **routes** an input from the **root** to a **prediction**.

Toy Example: Classification Tree

Used simple rules for routing (test a single feature) and for prediction (all inputs reaching a leaf will get have the same label predicted)





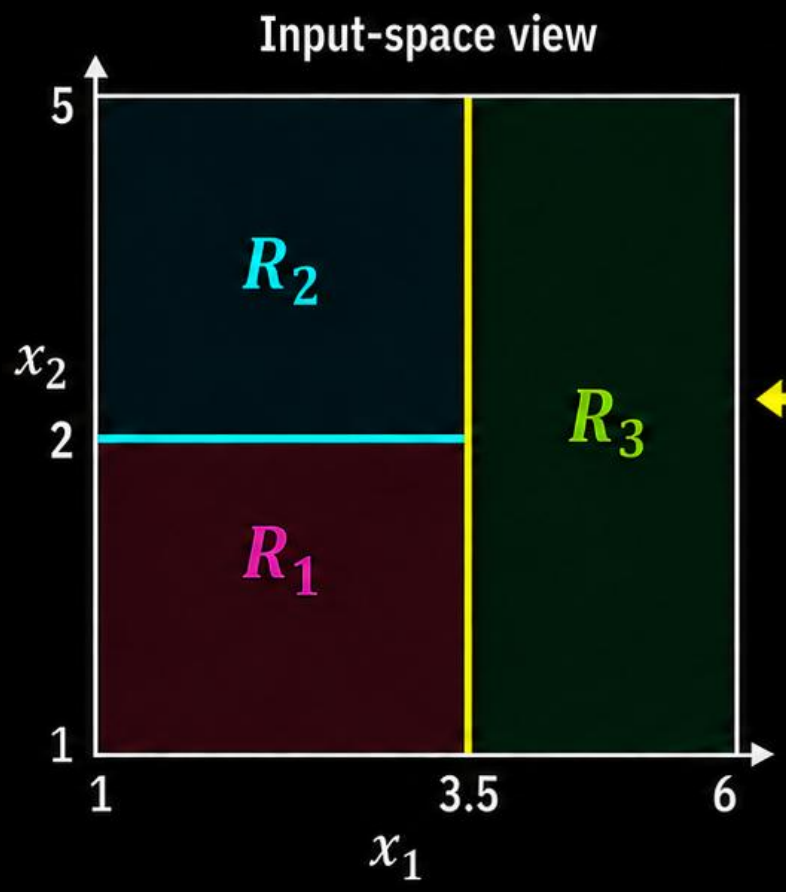


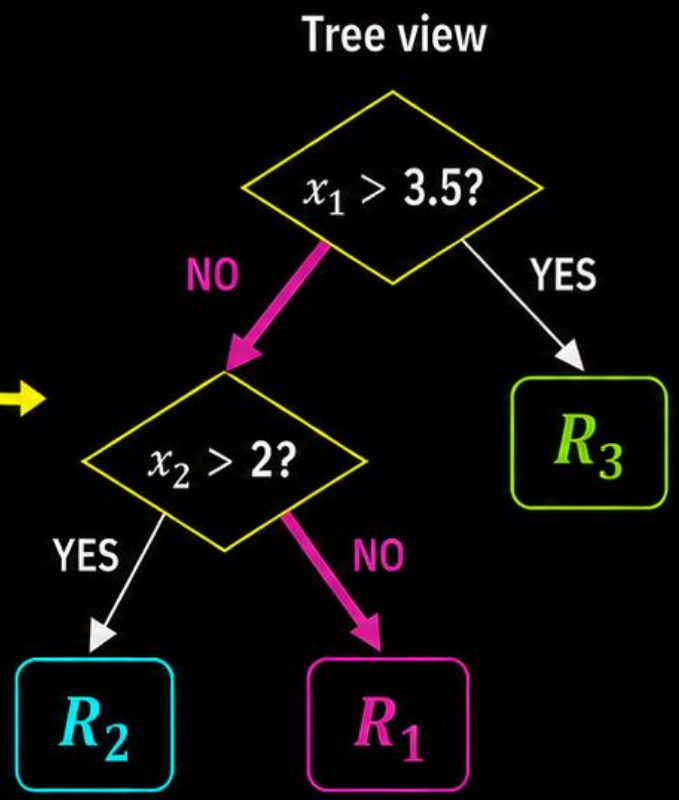
Each leaf node can be thought of as keeping a subset of training data (falling in the region represented by that leaf node)

A leaf corresponds to one region of the input space.

The tree locates a region — the leaf makes the class prediction.

A Tree Partitions the Input Space into Regions





Piecewise-constant view

$$f(\boldsymbol{x}) = \sum_{l} c_l \, \mathbb{I}[\boldsymbol{x} \in \boldsymbol{R}_l]$$

Classification: c_l = class

Regression: c_l = real-valued output

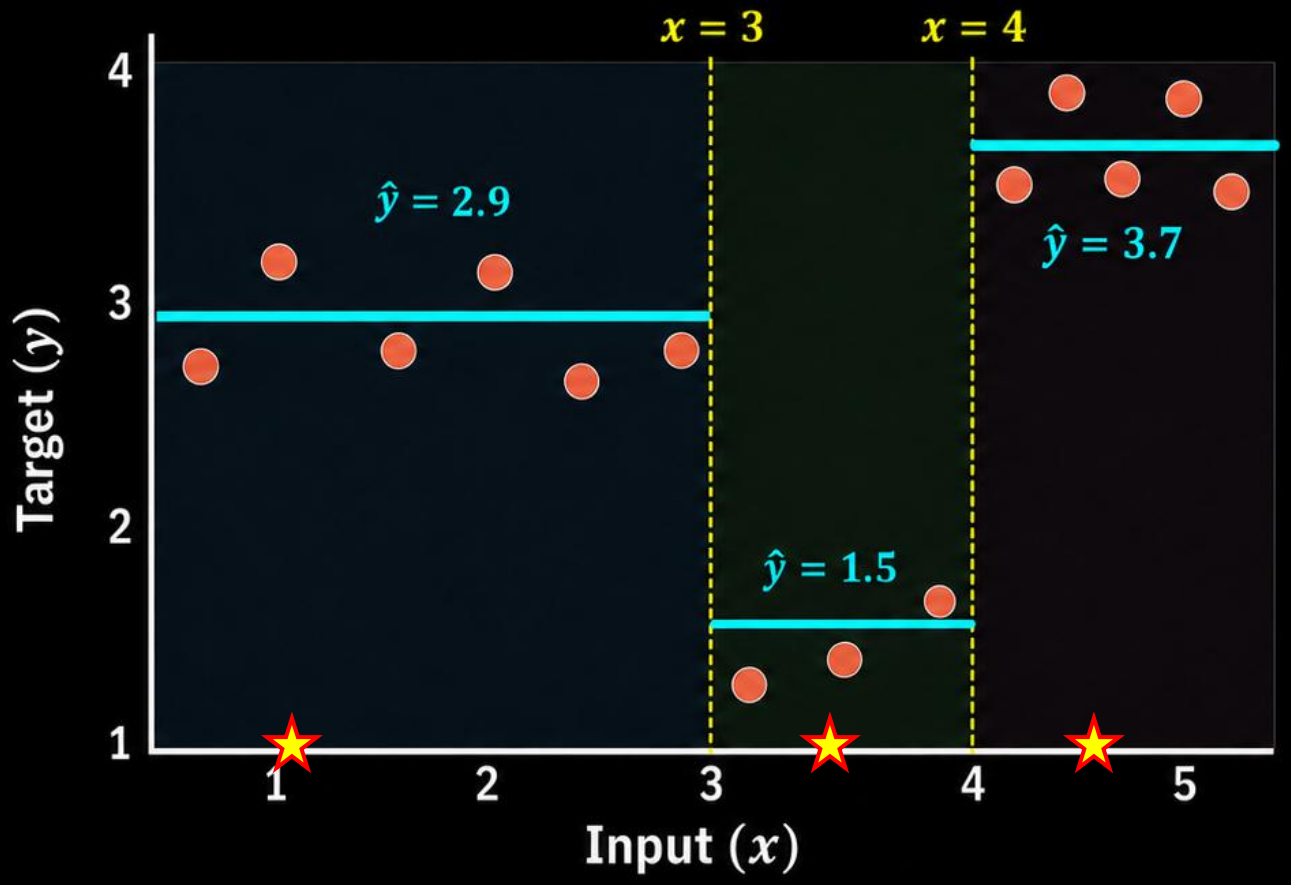
One leaf ↔ **one** region ↔ **one** prediction

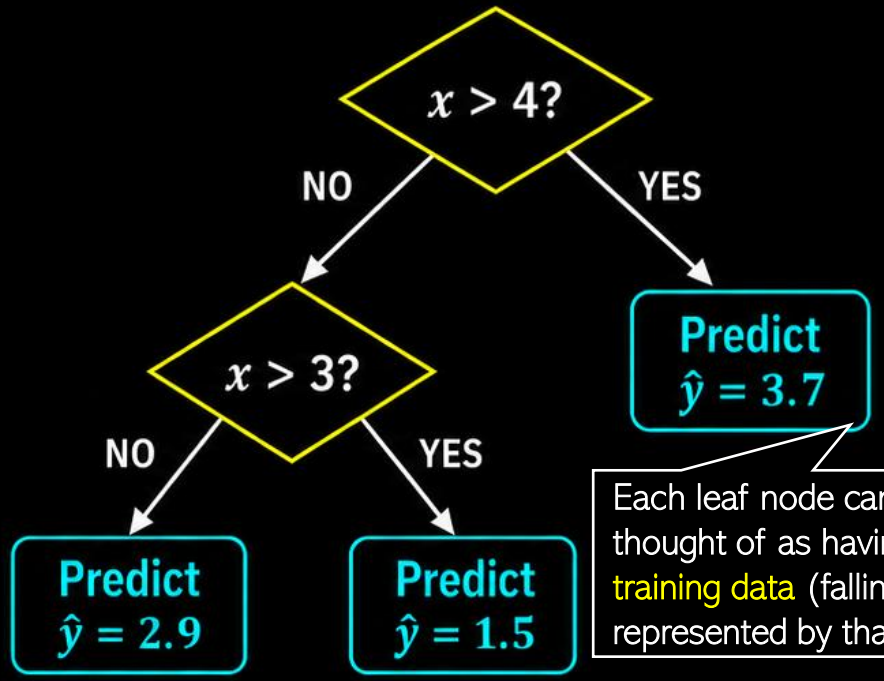
A root-to-leaf path defines a region where prediction is simple.

Toy Example: Regression Tree

Used simple rules for routing (test a single feature) and for prediction (all inputs reaching a leaf will get have the same real-valued output predicted)







Each leaf node can also be thought of as having a subset of training data (falling in the region represented by that leaf node)

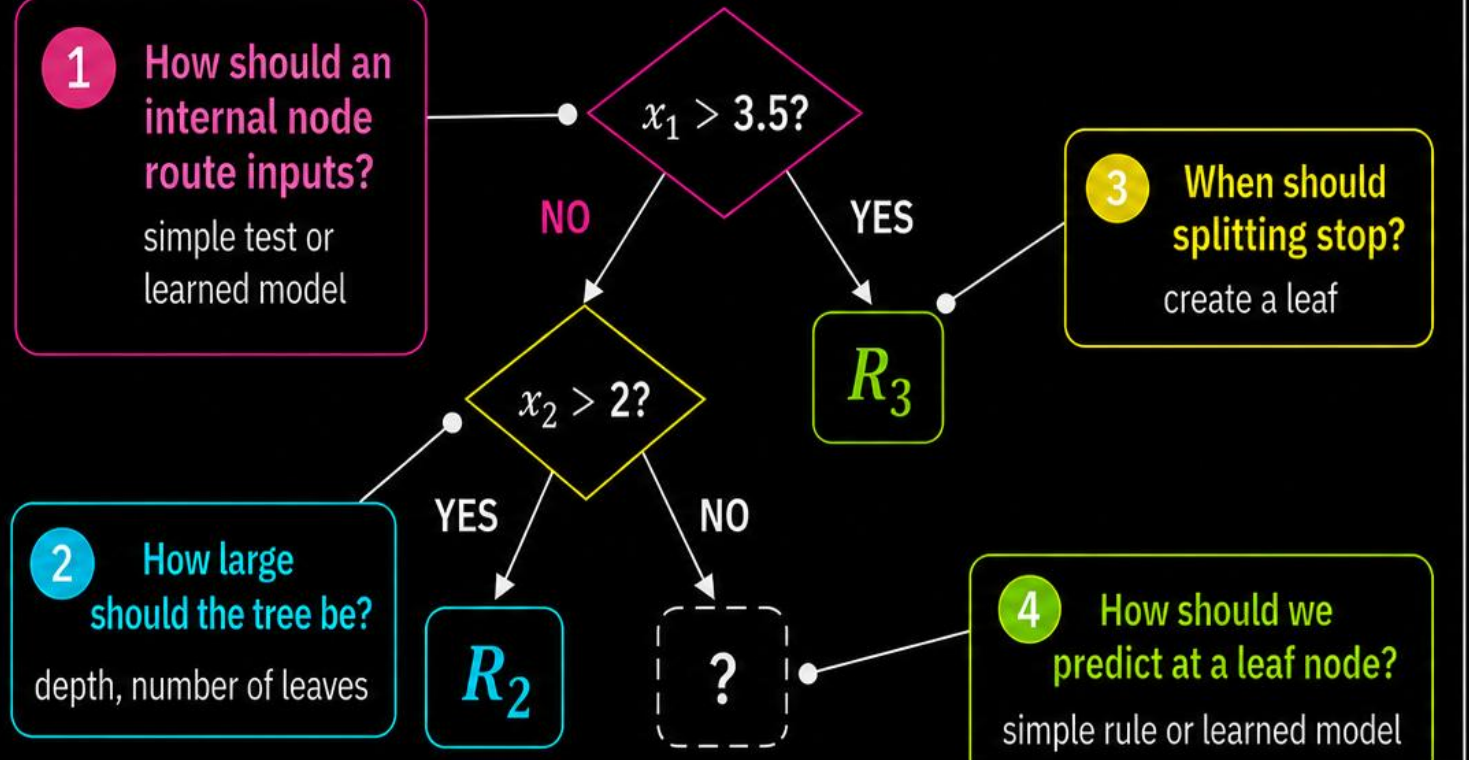
Standard regression leaf predicts the **mean target** in its region

Each leaf predicts one numerical value for its region.

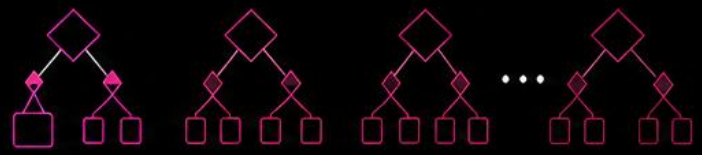
What Must A Decision Tree Learn?

Four design considerations

1. **How should an internal node route inputs?** simple test or learned model
2. **How large should the tree be?** depth, number of leaves
3. **When should splitting stop?** create a leaf
4. **How should we predict at a leaf node?** simple rule or learned model



Searching over all possible trees is generally NP-hard



Greedy recursive construction

1. Choose the best available split
2. Partition the data
3. Repeat within each child

Stop → create a leaf

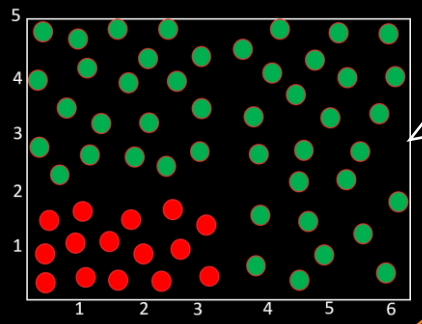
Next: What makes a split good?

Learn the tree recursively — one node at a time.

DT Learning: Split Criterion At Each Node



Chosen split criterion at any node should ensure high homogeneity/purity of its children nodes (averaged over all children)



All training examples at the root node

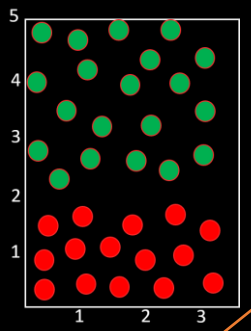
a split criterion

A good criterion: $x_1 > 3.5$?

NO

YES

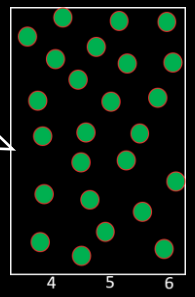
The group is not pure yet, can consider splitting



a split criterion

A good criterion: $x_2 > 2$?

If the group is pure, there is no need to split any further

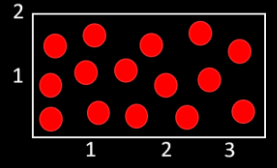


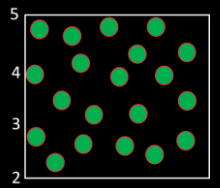
Predict Green

NO

YES

Predict Red

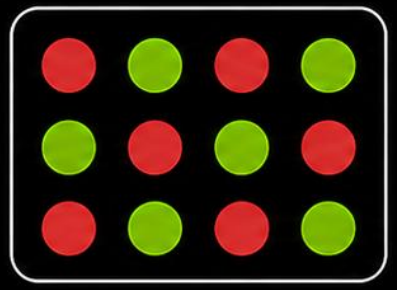




Predict Green

Notion of Purity/Homogeneity

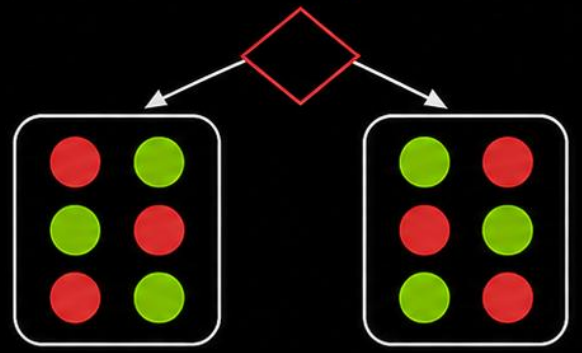
Before splitting



Mixed classes

HIGH IMPURITY

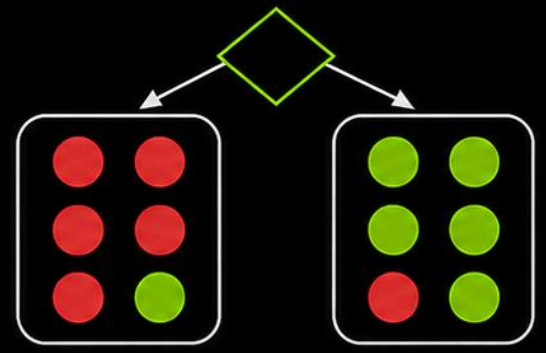
Weak split



still mixed still mixed

Small impurity reduction

Strong split



nearly pure nearly pure

Large impurity reduction

Classification

A good child node should ideally have **one label dominate** other labels.

Choose the split with the largest reduction in impurity.

Regression

A good child node should ideally have **all inputs** having **similar response values.**

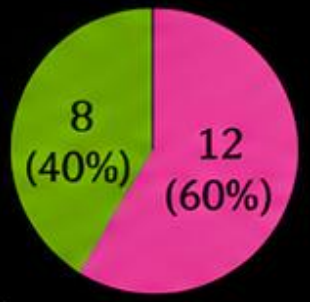
A good split makes the child nodes more homogeneous than the parent.

Purity/Homogeneity of a set S of Points

1. Classification (predicting a class)

Impurity $I(S)$ can be:

- Gini impurity:
  $1 - \sum_{k=1}^{K} p_k^2$
- Entropy:
  $- \sum_{k=1}^{K} p_k \log p_k$



Class 1 (Red): 12 of 20 (60%)

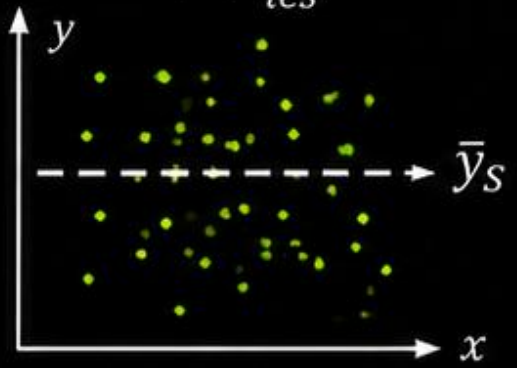
Class 2 (Green): 8 of 20 (40%)

Lower impurity ⇔ purer (one class dominates).

2. Regression (predicting a real value)

Impurity $I(S)$ = variance (MSE):

$$I(S) = \frac{1}{|S|} \sum_{i \epsilon S} (y_i - \bar{y}_S)^2$$



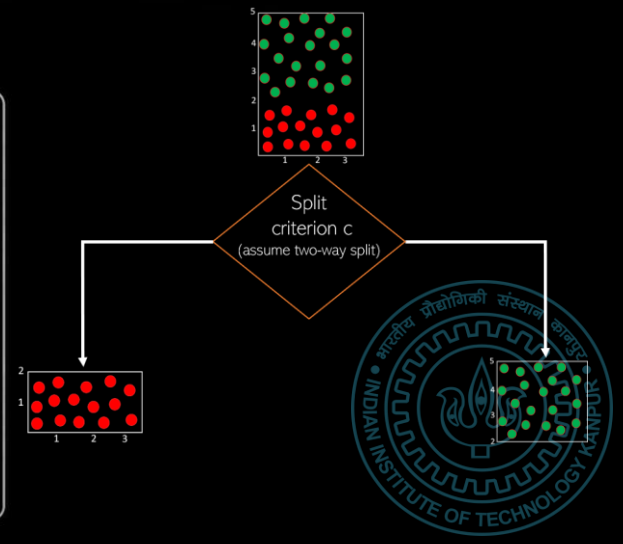
Lower variance means the target values in the set are more similar.

For the set S, from among all splitting criteria, pick the split criterion c that yield the largest gain (reduction in impurity after split)

Impurity reduction (Gain)

$$\textbf{Gain}(c) = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$$

- $I(S)$: impurity of parent set S
- $\frac{|S_L|}{|S|} I(S_L)$: weighted impurity of left child S_L
- $\frac{|S_R|}{|S|} I(S_R)$: weighted impurity of right child S_R



Possible Node Splitting Criteria

- A splitting node basically acts as a classifier or router
- Simple splitting criteria are most widely used
  - Testing the value of a single feature – for real-valued feature, it is a threshold based test (very fast to check)
- More sophisticated criteria also possible (but less commonly because difficult to find and/or slower to check), e.g.,
  - Use a combination of multiple features
  - Use a classifier (e.g. LwP) built using training data coming at that node

DT implementations prefer splitting criteria that are fast

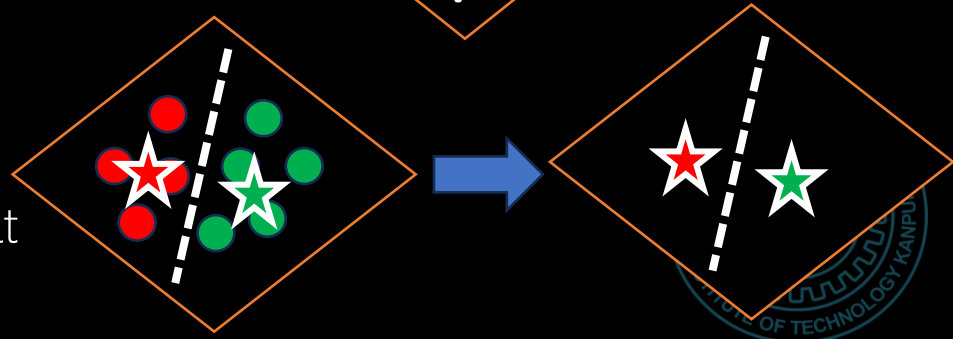


Here t denotes a threshold that we have to choose (will later discuss how to choose)

$x_1 \leq t$?

Or a more general linear combination of all input features, e.g., $w^T x \leq 0$

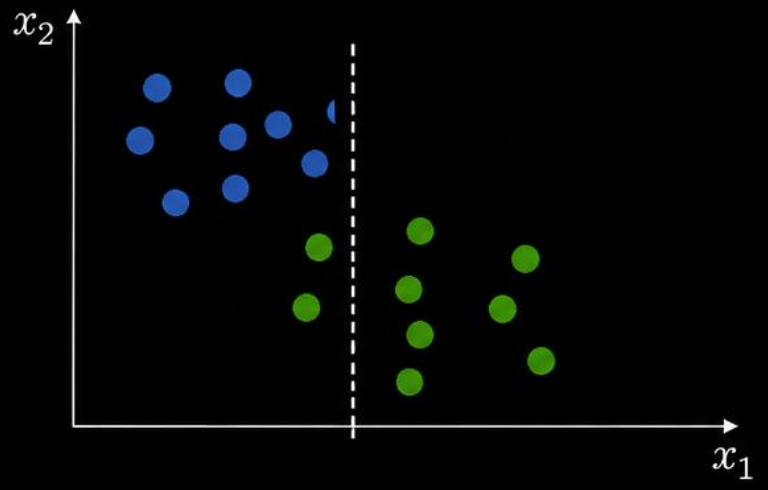
$x_1 \leq t_1$ and $x_2 \leq t_2$?



Decision Trees: Axis-Aligned vs Oblique Splits

1) Axis-Aligned Split (tests a single feature)

Split is parallel to one of the feature axes
(tests a single feature using a threshold).

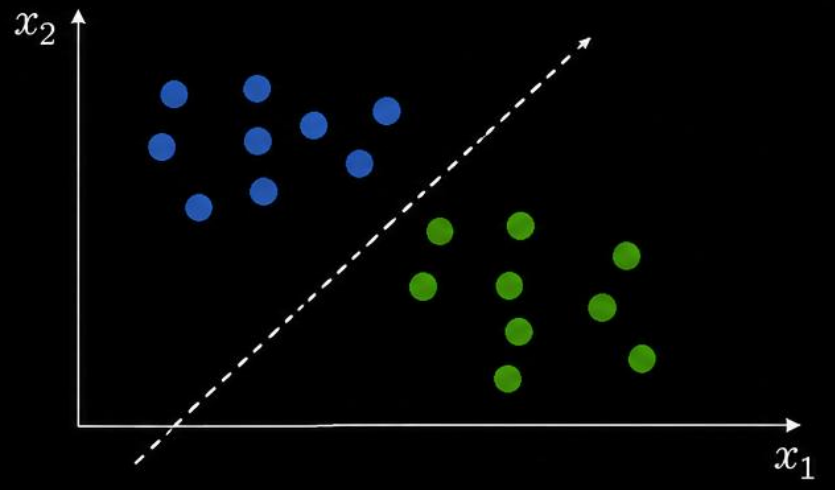


Left child: $x_1 \leq t$

Right child: $x_1 > t$

2) Oblique Split (tests a linear combination)

Split is not parallel to the axes
(tests a linear combination of features).



Left child: $w^{\mathrm{T}}x \leq 0$

Right child: $w^{\mathrm{T}}x > 0$



Axis-aligned split tests a single feature with a threshold (e.g., $x_1 \leq t$).

Oblique split tests a linear combination of features (e.g., $w^{\mathrm{T}}x \leq 0$).

An Example of Split using Real-Valued Feature

For each feature, sort the examples and try candidate thresholds between consecutive values.
Evaluate the impurity reduction ($Gain$) for each and keep the best.

Classification example
(3 classes)
- Class 1 (C_1)
- Class 2 (C_2)
- Class 3 (C_3)

1 Sort the examples by feature x_j

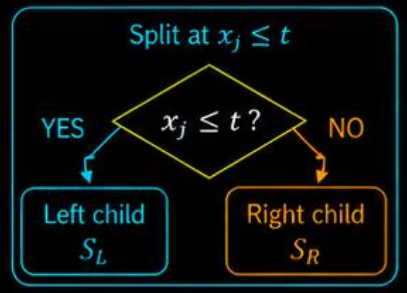
Values of x_j	1.2	1.8	2.7	3.1	3.6	4.2	4.8	5.3
Class label	●	●	●	●	●	●	●	●

Candidate thresholds (midpoints): 1.50, 2.25, 2.90, 3.35, 3.90, 4.50, 5.05

Parent set S (8 examples)
Class counts $(C_1, C_2, C_3) = (2, 3, 3)$
Gini impurity of S

$$I(S) = 1 - \sum_c p_c^2 = 1 - \left[\left(\tfrac{2}{8}\right)^2 + \left(\tfrac{3}{8}\right)^2 + \left(\tfrac{3}{8}\right)^2\right] = 0.65625$$

2 For each candidate threshold t, split the data and compute the Gini reduction ($Gain$)



Threshold t	1.50	2.25	2.90	3.35	3.90	4.50	5.05
Left / Right counts (n_L / n_R)	1 / 7	2 / 6	3 / 5	4 / 4	5 / 3	6 / 2	7 / 1
Left child S_L (C_1, C_2, C_3)	(1, 0, 0)	(1, 1, 0)	(2, 1, 0)	(2, 2, 0)	(2, 2, 1)	(2, 3, 1)	(2, 3, 2)
Right child S_R (C_1, C_2, C_3)	(1, 3, 3)	(1, 2, 3)	(0, 2, 3)	(0, 1, 3)	(0, 1, 2)	(0, 0, 2)	(0, 0, 1)
Gini reduction Gain(t)	0.1205	0.0729	0.1896	0.2188	0.0896	0.0313	0.0848

(All Gini values and gains computed using fractions; rounded to 4 decimals.)

Gini reduction (Gain)

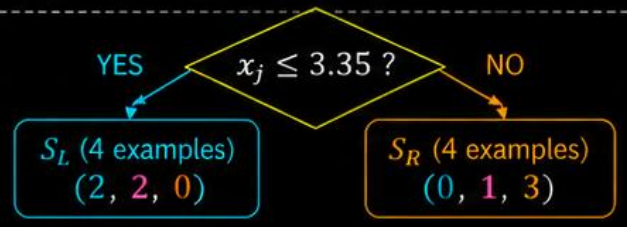
$$Gain(t) = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$$

where $I(\cdot)$ is Gini impurity

(Lower is better for impurity, Higher is better for Gain.)

3 Choose the threshold with the largest gain.

Best threshold:
$t^* = 3.35$
(Gain = 0.2188)



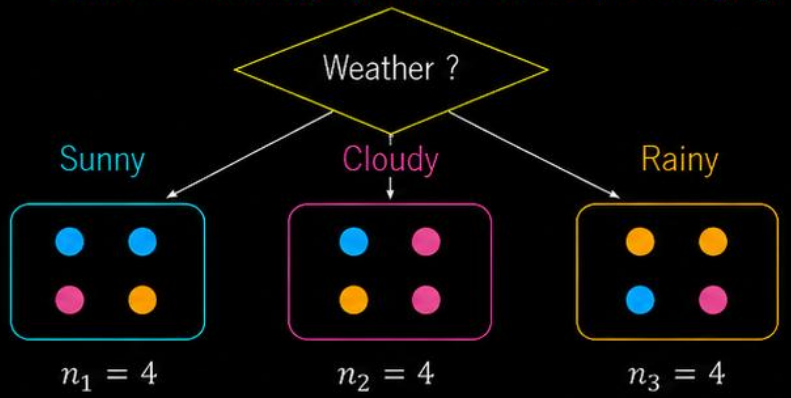
Repeat this for all features and select the split with the largest Gain overall.

An Example of Split using Categorical Feature

For a categorical feature, we consider the split criteria based on the finite possible values this feature takes.

3-class problem (3 labels)
Class 1 Class 2 Class 3

Option 1: Multiway split (one branch per category)



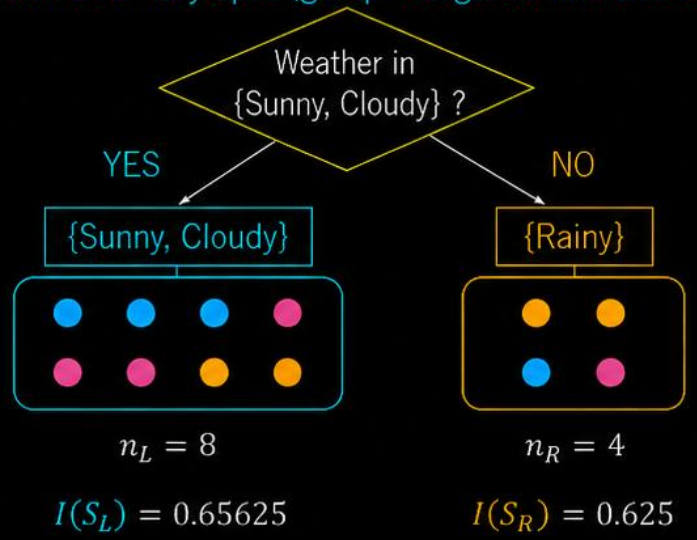
$$Gain = I(S) - \frac{4}{12} I(S_1) - \frac{4}{12} I(S_2) - \frac{4}{12} I(S_3)$$

General Gain formula

$$Gain = I(S) - \sum_k \frac{|S_k|}{|S|} I(S_k)$$

$I(\cdot)$ = impurity

Option 2: Binary split (group categories into two sets)



$$Gain = I(S) - \frac{8}{12} I(S_L) - \frac{4}{12} I(S_R)$$

We try all reasonable partitions (or use multiway split) and choose the one with the largest Gain.

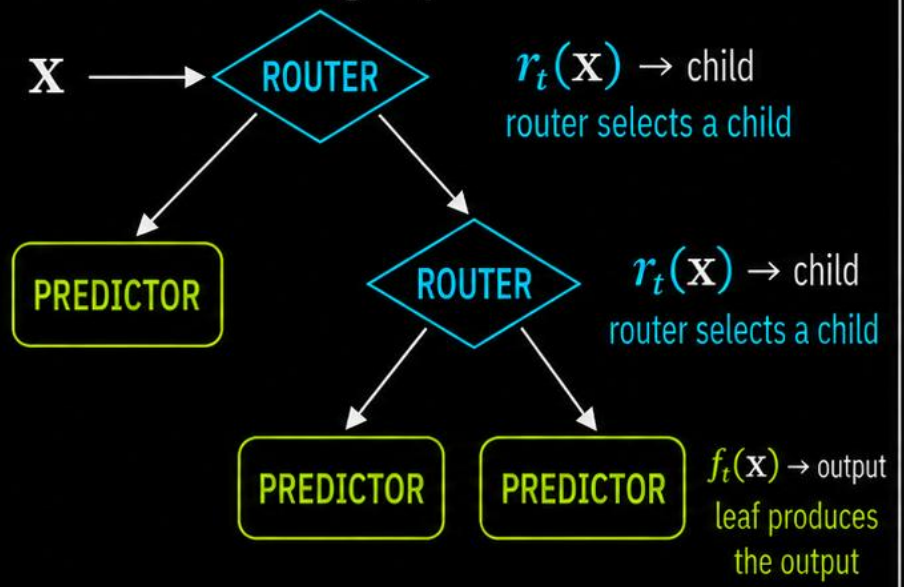
Same objective. Different split candidates.

Categorical features are naturally handled by trees.

Decision Tree as a General Framework

The tree specifies **where computation happens** — not **what model** each node must use.

Routing + prediction



Framework = routing + prediction

Component	Standard choice	More general choice
Internal node (router)	Test a single feature's value	Oblique split or learned router
Classification leaf (predictor)	Majority class	Local classifier
Regression leaf (predictor)	Mean target	Local regressor

A learned leaf model is trained on the examples routed to that leaf.

Most commonly used implementations: single-feature tests + simple leaf predictions.

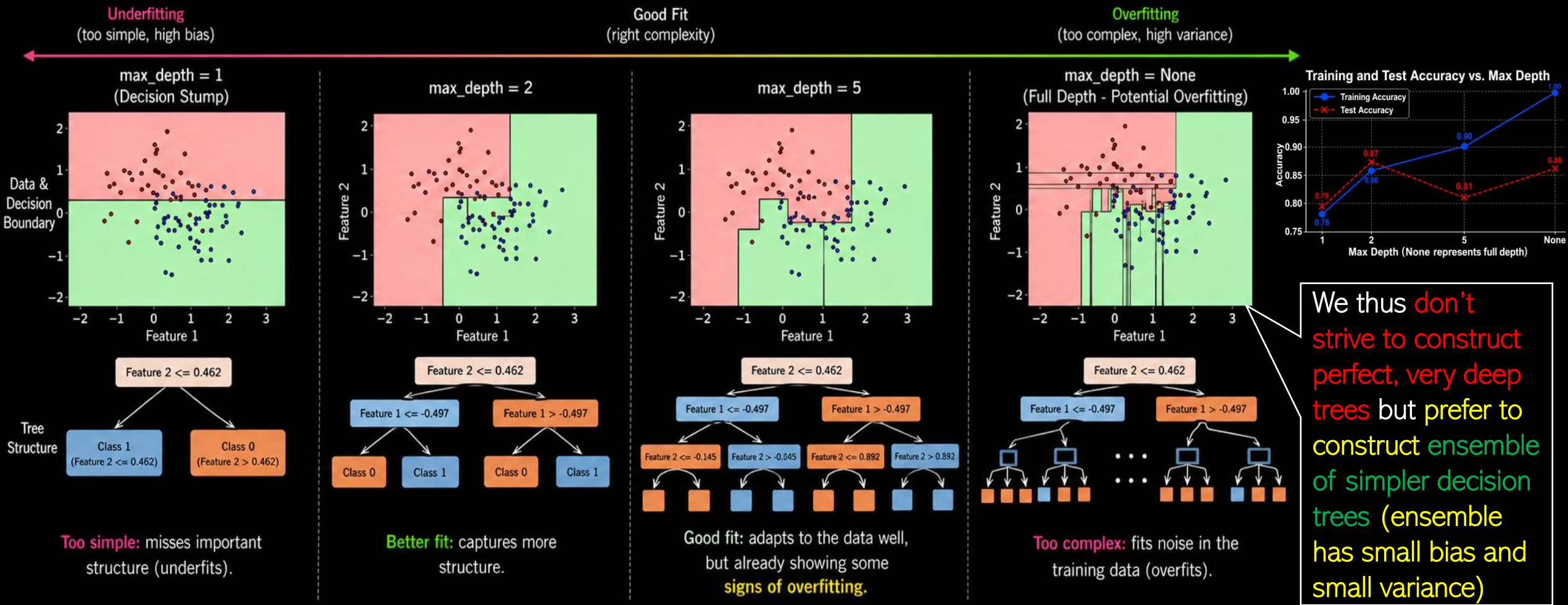
Relying on a Single DT?

Stable = Tree won't change much to minor changes (addition, deletions, perturbations) in training data

Unstable = Tree shape may change drastically even to minor changes in training data

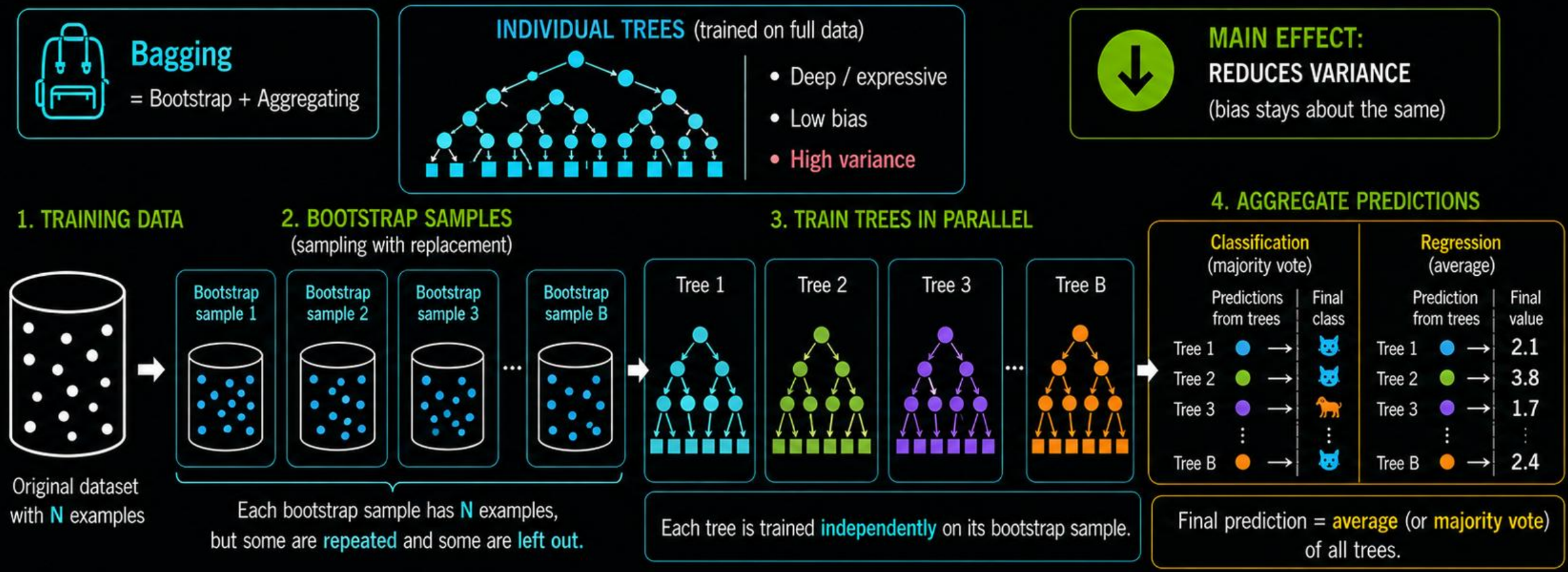
High Bias DT: Too simple, doesn't fit training data that well, but is stable

High Variance DT: Too complex, fits training data almost perfectly, but unstable



We thus don't strive to construct perfect, very deep trees but prefer to construct ensemble of simpler decision trees (ensemble has small bias and small variance)

Ensembles of Trees – Forests



KEY IDEA

Different bootstrap samples → different trees.
Averaging cancels out their high variance.

Random Forest = Bagging
+ random subset of features
considered at each split.

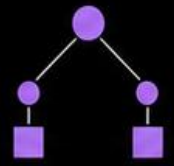
POPULAR METHODS

Random Forest | Extra Trees

Ensembles of Trees - Boosting

Boosting
= Combine weak learners sequentially

INDIVIDUAL TREES (weak learners)



- Simple / shallow (e.g., depth-1 or depth-2)
- Higher bias
- Lower variance

MAIN EFFECT:
REDUCES BIAS
(variance usually stays similar)

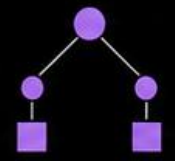
Training Data



Dataset (X, y)

Used by the first weak learner.

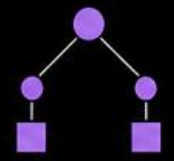
Train Weak Tree 1



$h_1(x)$

Learns from the data. Makes many errors.

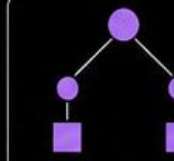
Update & Train Tree 2



$h_2(x)$

Focuses more on the mistakes of current ensemble (h_1).

Update & Train Tree 3



$h_3(x)$

Focuses more on the remaining mistakes of current ensemble ($h_1 + h_2$).

Update & Train Tree M



$h_M(x)$

Focuses more on the remaining mistakes of current ensemble.

Final Ensemble

Strong Model

$$F_M(x) = \sum_{m=1}^{M} \alpha_m h_m(x)$$

Weighted sum of all weak trees.

Key Idea: Many weak learners → one strong learner.

HOW IT WORKS (INTUITION)

- Each tree tries to fix the mistakes of the current ensemble.
- Misclassified (or poorly predicted) examples get more focus.
- Errors are reduced gradually; the ensemble gets stronger.

WHY IT WORKS

- Correcting errors reduces bias.
- Combining many weak models yields a very powerful model.

POPULAR METHODS

AdaBoost | Gradient Boosting | XGBoost | LightGBM | CatBoost

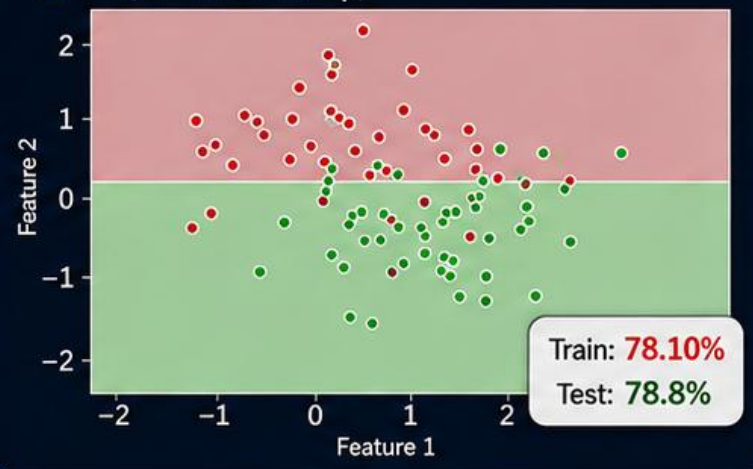
Individual vs Ensemble of Trees: An Illustration

INDIVIDUAL TREES (Single Models)

1 Decision Tree: max_depth=None (Full Depth - Potential Overfitting)



2 Decision Tree: max_depth=1 (Decision Stump)

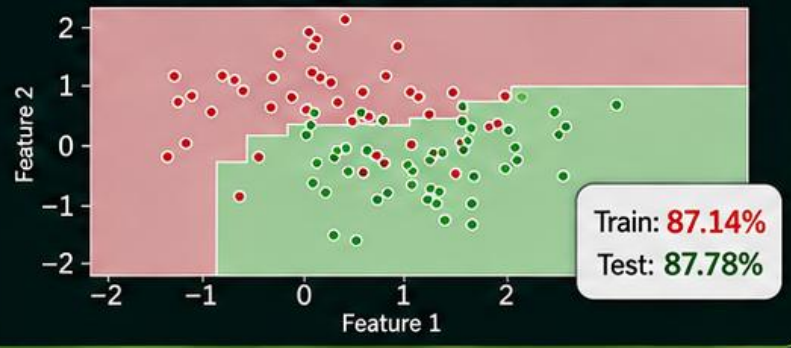


3 Decision Tree: max_depth=5

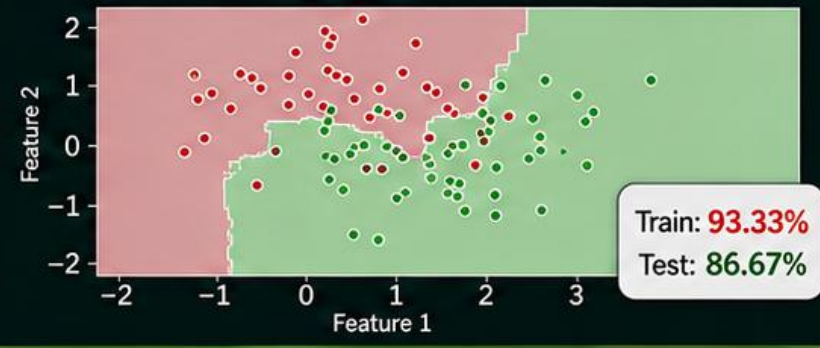


ENSEMBLE MODELS (Combined Trees)

4 Gradient Boosting Classifier 50 Trees, max_depth=1 (Decision Stump)



5 Random Forest Classifier 100 Trees, max_depth=5



Key Takeaway

Individual trees can have high variance or high bias.

Ensembles strike a better balance, improving generalization.

DTs and Their Ensembles: Summary & Outlook

What We Learned

1. A decision tree recursively partitions the feature space to make predictions.
2. Splits can be axis-aligned (single feature threshold) or oblique (linear combinations).
3. Good splits reduce impurity (e.g., Gini impurity) the most.
4. Trees are easy to interpret but can overfit if grown too deep.
5. Ensembles of trees (bagging, boosting) achieve high accuracy and robustness.

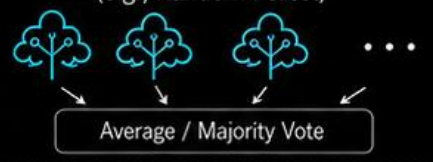
Why Decision Trees Are Useful

- **Interpretability:** human-readable rules and clear reasoning.
- **Transparency & Trust:** easy to audit, explain, and debug.
- **Handles Mixed Data:** works with numerical, categorical, and missing values.
- **Minimal Preprocessing:** little feature scaling or engineering needed.
- **Feature Importance:** highlights what matters.

Tree Ensembles: More Power

Bagging

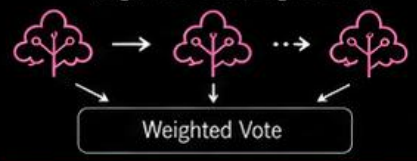
(e.g., Random Forest)



- Reduces variance
- More stable
- Great for accuracy

Boosting

(e.g., XGBoost, LightGBM)



- Reduces bias (and variance)
- Captures complex patterns
- State-of-the-art on tabular data

The Data Divide: Deep Learning vs. Tree Ensembles

Deep Learning Rules Unstructured Data



Images (pixels) | Text (sequences) | Audio / Video (signals)

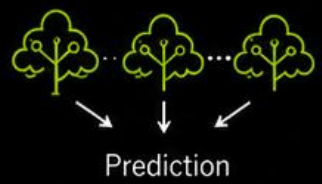
Strength: learns hierarchical representations automatically.

Best for: vision, language, speech, and other unstructured data.

VS.

Tree Ensembles Rule Structured (Tabular) Data

Age	Income	City	Product	...	Target
25	50K	A	Yes	...	1
40	80K	B	No	...	0
35	60K	A	Yes	...	1
...	...	...	...		...



Strength: excels on tabular/structured data.

Best for: business, finance, healthcare, risk, forecasting, operations.

Why They Matter in 2026

Still State-of-the-Art on Tabular Data
Tree ensembles (e.g., XGBoost, LightGBM, CatBoost) remain the top choice in competitions and real-world tabular problems.

Robust & Reliable
Work well with limited data, outliers, and noisy features.

Interpretable AI
Essential for regulated domains (finance, healthcare, legal, gov).

Fast & Efficient
Training and inference are efficient and production-friendly.

Complements Deep Learning
Use the right tool for the right data—together they solve real-world problems.

Bottom line: Decision Trees give us understanding. Tree Ensembles give us performance. Together give us impacts.
Understand with trees. Predict with ensembles. Choose the right model for the right data.